

Audit-Report Noders go-daml SDK & Wallet 08.2026

Cure53, Dr.-Ing. M. Heiderich, Dr. A. Pirker, Dipl.-Ing. C. Steinauer

Index

[Introduction](#)

[Scope](#)

[Identified Vulnerabilities](#)

[NOD-01-003 WP1: ZIP bomb DoS caused by no size limit \(Medium\)](#)

[NOD-01-005 WP1: Path traversals in scan-proxy client \(Low\)](#)

[NOD-01-006 WP1: Request forgery in SDK's network via redirects \(Low\)](#)

[NOD-01-008 WP2: Memory exhaustion DoS via unbounded streams \(Medium\)](#)

[NOD-01-010 WP1: Improper handling of TLS config parameters \(Medium\)](#)

[NOD-01-012 WP1: Float64 precision loss in JSON unmarshalling \(Medium\)](#)

[NOD-01-013 WP1: Inconsistency in BigInt scaling of JSON codec \(Medium\)](#)

[Miscellaneous Issues](#)

[NOD-01-001 WP1: Missing support for mTLS in SDK \(Low\)](#)

[NOD-01-002 WP2: Float64 precision issue in unreferenced functions \(Info\)](#)

[NOD-01-004 WP1: Unbound recursion in response decoding \(Info\)](#)

[NOD-01-007 WP1: Potential infinite loop in ledger API services \(Info\)](#)

[NOD-01-009 WP2: Data unreadability due to no query offset \(Info\)](#)

[NOD-01-011 WP1: Ledger and admin gRPC connections use same config \(Low\)](#)

[NOD-01-014 WP1: No length check in topology-manager could lead to panic \(Info\)](#)

[NOD-01-015 WP1: Import of vulnerable packages and dependencies \(Info\)](#)

[Conclusions](#)

Introduction

“We build cool applications, validate blockchains, provide highly reliable infrastructure and grow communities. Investors earn passive income using our non-custodial staking”

From <https://noders.team/>

This report describes the results of a security assessment of the Noders complex, focusing on the Noders *go-daml* SDK and Wallet. The project, which included a penetration test and a dedicated source code audit, was conducted by Cure53 in August 2026.

The audit, registered as *NOD-01*, was requested by Noders LLC in July 2026 and was then scheduled to start the following month to give both sides time to prepare. The project marks the first-time cooperation between Cure53 and Noders.

In terms of the exact timeline and specific resources allocated to *NOD-01*, the Cure53 team has completed their research in CW32 and CW33. In order to achieve the expected coverage of the components in scope, a total of eleven days were invested. A team consisting of three senior testers was formed and assigned to the preparation, execution, documentation, and delivery of this project.

For optimal structuring and tracking of tasks, the assessment was divided into two separate work packages (WPs):

- **WP1:** White-box source code audits & reviews of *go-daml* core SDK
- **WP2:** White-box code audits & reviews of *go-daml* Wallet & Amulet standard

As the titles of the WPs indicate, the white-box methodology was used for examination approaches across all components in scope. Relevant Canton URLs, credentials, documentation, as well as all further means of access required to complete the tests were provided to the Cure53 testing team. Likewise, all sources corresponding to the test targets were shared to ensure that the project could be executed in accordance with the agreed framework.

The project could be carried out without any major issues. To facilitate a smooth transition into the testing phase during the summer season, all preparations were completed in late July, namely in CW31. Throughout the engagement, communications were conducted through a private, dedicated, and shared Slack channel. Stakeholders - including Cure53 testers and the internal staff from Noders - were able to participate in discussions in this space.

Cure53 did not need to ask many questions, and the quality of all project-related interactions was consistently excellent. Frequent status updates on the examination and emerging findings were paired with live-reporting, which was done through Markdown files and executed over the mentioned Slack channel.

Continuous communication contributed positively to the overall results of this project. Significant roadblocks were avoided thanks to clear and careful preparation of the scope, as well as through subsequent support.

The Cure53 team achieved very good coverage of the WP1-WP2 objectives and scope. Of the fifteen security-related discoveries, seven were classified as security vulnerabilities and eight were filed as general weaknesses with lower exploitation potential. On the one hand, the spotted flaws were assessed as posing relatively manageable risks in terms of impact, with no single finding exceeding the *Medium* severity score. On the other hand, Cure53 believes that the codebase is in a notably fragile state.

In particular, the testers recognize that the Noders project has been built on sound cryptographic fundamentals and maintains good credential hygiene. Its general architecture reflects strong core practices, yet the presence of multiple architectural oversights strongly indicates that security and code maintenance were not consistently prioritized throughout the development lifecycle. This gap between strong foundational cryptographic choices and surrounding implementation flaws render the system exposed and vulnerable to various risks.

The core implementation demonstrated significant susceptibility to malicious external inputs, especially in upstream communications and processing of data structures. Serious concerns were raised regarding the handling of numeric conversions and data precision. In this realm, subtle flaws could easily compromise financial accuracy and overall application logic.

Ultimately, while the absence of immediate threats - like *Critical* or *High*-impact vulnerabilities - provides a temporary baseline, the cumulative weight of structural and handling issues is a problem. In fact, it currently forges a fragile surface that requires rather urgent comprehensive hardening.

The following sections first describe the scope and key test parameters, as well as how the two work packages were structured and organized.

Next, all findings are presented as vulnerabilities and as miscellaneous flaws. The problems are then discussed chronologically within each category. In addition to technical descriptions, PoC and mitigation advice is provided where applicable.

The report ends with general conclusions relevant to this summer 2026 project. Based on the test team's observations and the evidence collected, Cure53 elaborates on the overall impressions and reiterates the verdict. The final section also includes tailored hardening recommendations for the Noders complex, specifically the *go-daml* core SDK, Wallet and Amulet standard components.

Scope

- **White-box code audits & reviews of *go-daml* SDK, Wallet & Amulet standard**
 - **WP1:** White-box source code audits & reviews of *go-daml* core SDK
 - **Source code repository:**
 - **URL:**
 - <https://github.com/noders-team/go-daml>
 - **Branch:**
 - main
 - **Commit:**
 - 992d6d8b1dd1b8cfc532b55163b45c57bfbf1b94
 - **Documentation**
 - <https://github.com/noders-team/go-daml/tree/main/docs/dev>
 - **WP2:** White-box code audits & reviews of *go-daml* Wallet & Amulet standard
 - **Source code repository:**
 - **URL:**
 - <https://github.com/noders-team/go-daml>
 - **Branch:**
 - main
 - **Commit:**
 - 992d6d8b1dd1b8cfc532b55163b45c57bfbf1b94
 - **Folder:**
 - /wallet
 - **Documentation**
 - <https://github.com/noders-team/go-daml/tree/main/docs/dev>
 - **Credentials for test-users:**
 - Canton URLs
 - <https://ledger-api-grpc.participant.canton-audit-01.devnet.naas.noders.services>
 - <https://ledger-api-json.participant.canton-audit-01.devnet.naas.noders.services>
 - <https://validator-api-http.validator.canton-audit-01.devnet.naas.noders.services>
 - **OIDC** – <https://keycloak.naas.noders.services/realms/canton-audit>
 - Client: web-app-ui-canton-audit-01-devnet
 - Username: christoph@rs.cure53.de
 - Username: apirker@cure53.de
 - Username: julian@cure53.de
 - **Test-supporting material was shared with Cure53**
 - **All relevant sources were shared with Cure53**

Identified Vulnerabilities

The following section lists all vulnerabilities and implementation issues identified during the testing period. Notably, findings are cited in chronological order rather than by degree of impact, with the severity rank offered in brackets following the title heading for each vulnerability. Every ticket has a unique identifier (e.g., *NOD-01-001*) to facilitate any follow-up correspondence.

NOD-01-003 WP1: ZIP bomb DoS caused by no size limit (*Medium*)

The *ScanProxyClient* in *pkg/client/scanproxy.go* does not enforce any size limits when processing HTTP responses from the scan-proxy server. On the success path (HTTP 2xx), response bodies are decoded directly without wrapping the body in a size-limiting reader. On the error path (HTTP 4xx/5xx), *io.ReadAll(resp.Body)* is used to read the entire response body into memory, again without any size restrictions in place.

In a scenario where the scan-proxy returns an unexpectedly large response, the SDK process could allocate unbounded memory. While the *http.Client* is configured with a 30-second timeout, this does not sufficiently constrain the volume of the processed data.

By default, the Go *http.Client* automatically decompresses *gzip* responses unless they are explicitly disabled. Consequently, a compromised scan-proxy could deliver a highly compressed payload within this 30-second window. Upon decompression, this payload would rapidly exhaust system memory, potentially leading to an application crash.

The application shown below can be used to demonstrate the issue. It uses the same primitives as also the scan-proxy implementation of the SDK, i.e., it creates a new context using *http.NewRequestWithContext* and executes the request through *http.Client* by only using a time-out. Consequently, the entire response is read utilizing *io.ReadAll* directly from the body of the response.

PoC application:

```
package main

import (
    "context"
    "fmt"
    "io"
    "net/http"
    "strings"
    "time"
)

func main() {
```

```
req, _ := http.NewRequestWithContext(context.Background(),
    "POST",
    "http://127.0.0.1:8083/post-post",
    strings.NewReader("hola"))

req.Header.Set("Content-Type", "application/x-www-form-urlencoded")

httpClient := &http.Client{Timeout: 10 * time.Second}
resp, err := httpClient.Do(req)
if err != nil {
    return
}
defer resp.Body.Close()

b, err := io.ReadAll(resp.Body)
fmt.Println(string(b))
}
```

To create the server hosting a ZIP bomb, the command sequence indicated below was used. The first command creates a file of size 2GB and compresses it into the file *out.gz*. The second command returns the file in an HTTP response to any request to port 8083. Running the application above after starting the server below will increase the application's memory to more than 2GB, thereby demonstrating the problem.

Command sequence:

```
yes A | head -c 2000000000 | gzip > out.gz
(printf 'HTTP/1.1 200 OK\r\n'; printf 'Content-Type: text/plain\r\n';
printf 'Content-Encoding: gzip\r\n'; printf 'Content-Length: %s\r\n' "$(wc
-c < out.gz)"; printf '\r\n'; cat out.gz; ) | nc -l 8083
```

Affected file:

pkg/client/scanproxy.go

Affected code:

```
[...]
func NewScanProxyClient(baseURL string, provider auth.TokenProvider)
*ScanProxyClient {
    logger := log.Logger.With().Str("component", "scan-proxy-
client").Logger()

    return &ScanProxyClient{
        baseURL:      baseURL,
        httpClient:    &http.Client{Timeout: 30 * time.Second},
        tokenProvider: provider,
        logger:        logger,
    }
}
```

```

}

func (s *ScanProxyClient) do(ctx context.Context, method, path string,
body, result interface{}) error {
    [...]
    resp, err := s.httpClient.Do(req)
    if err != nil {
        return fmt.Errorf("failed to execute request: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode >= 400 {
        bodyBytes, _ := io.ReadAll(resp.Body)
        s.logger.Error().Int("status", resp.StatusCode).Str("body",
string(bodyBytes)).Msg("scan-proxy error response")
        return fmt.Errorf("HTTP error %d: %s", resp.StatusCode,
string(bodyBytes))
    }

    if result != nil {
        if err := json.NewDecoder(resp.Body).Decode(result); err !=
nil {
            return fmt.Errorf("failed to decode response: %w", err)
        }
    }
    return nil
}

```

Cure53 recommends wrapping *resp.Body* in an *io.LimitReader* with a reasonable upper-bound before processing. This would be consistent with the pattern already used in the Keycloak token provider. The new approach should be applied to both the success and error paths. Additionally, to further harden the mechanism against compression bombs, Cure53 advises disabling *gzip* support unless it is strictly required by the application.

NOD-01-005 WP1: Path traversals in scan-proxy client (*Low*)

During a source code review of the SDK it was noticed that the *ScanProxyClient* implementation exposes functions that send HTTP requests to an upstream server. When constructing the paths of such outbound requests, the SDK utilizes string interpolation without sanitizing the interpolated strings from traversal payloads such as *../* first. This results in a path traversal vulnerability.

If an application uses these vulnerable functions of the SDK without sanitizing the interpolated parameters, the SDK will consume the traversal payloads. As a consequence, the SDK calls unintended upstream HTTP endpoints. This can result in further, unspecified harm.

The file excerpt shown below demonstrates the issue. All three functions use a string parameter without sanitizing it in a string interpolation to obtain the final request's path.

Affected file:

go-daml/pkg/client/scanproxy.go

Affected code:

```
func (s *ScanProxyClient) GetTransferInstructionChoiceContext(ctx
context.Context, transferInstructionID, choicePath string)
(*TransferInstructionChoiceContext, error) {
    path := fmt.Sprintf(
        "/v0/scan-proxy/registry/transfer-instruction/v1/%s/choice-
contexts/%s",
        url.PathEscape(transferInstructionID),
        choicePath,
    )
    [...]
    if err := s.post(ctx, path, map[string]interface{}{}, &resp); err !=
nil {
        return nil, fmt.Errorf("failed to get transfer instruction
choice context: %w", err)
    }
    [...]
}
[...]
func (s *ScanProxyClient) GetTransferPreApprovalByParty(ctx
context.Context, party string) (*ScanContract, error) {
    path := fmt.Sprintf("/v0/scan-proxy/transfer-preapprovals/by-
party/%s", party)
    var resp TransferPreapprovalResponse
    if err := s.get(ctx, path, &resp); err != nil {
        return nil, fmt.Errorf("failed to get transfer preapproval:
%w", err)
    }
    [...]
}
[...]
func (s *ScanProxyClient) GetFeaturedAppByProvider(ctx context.Context,
providerParty string) (*ScanContract, error) {
    path := fmt.Sprintf("/v0/scan-proxy/featured-apps/%s", providerParty)
    var resp FeaturedAppResponse
    if err := s.get(ctx, path, &resp); err != nil {
        return nil, fmt.Errorf("failed to get featured app: %w", err)
    }
    [...]
}
```

Cure53 advises implementing a guard against path traversal payloads by normalizing the path. What is more, Noders should incorporate strict input data validations for the expected format of interpolated strings (e.g., alpha-numeric, letters, etc.).

NOD-01-006 WP1: Request forgery in SDK's network via redirects (*Low*)

On several occasions, the SDK sends requests to upstream services, such as Keycloak or scan-proxy. For that purpose, the SDK uses Golang's built-in HTTP client. This HTTP client, by default, follows redirects in case an upstream service responds with a redirection code such as 301 or 307. This also applies to internal/private IP addresses. As such, a request forgery vulnerability can be caused in applications using the SDK, provided that an upstream service gets compromised.

To be more specific, if an application using the SDK sends an upstream HTTP request to a compromised service, this service could redirect the request to an internal or private IP address of the application's network. Depending on the protections present in these internal/private services, more harm could be done.

Steps to reproduce:

1. Create a Go application which sends an HTTP request using *http.Client* together with *http.NewRequestWithContext* to the server shown in *Step 2*.

PoC Go application:

```
package main

import (
    "context"
    "net/http"
    "strings"
    "time"
)

func main() {
    req, _ := http.NewRequestWithContext(context.Background(),
        "POST",
        "http://127.0.0.1:8083/post-post",
        strings.NewReader("hola"))

    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")

    httpClient := &http.Client{Timeout: 10 * time.Second}
    httpClient.Do(req)
}
```

2. Create and start a NodeJS server which redirects the request from *Step 1* and outputs a log statement for each endpoint. An example is shown below.

PoC NodeJS server:

```
const express = require('express')
const app = express()

let port = 8083

app.use(express.raw({ type: '*/*', limit: '10mb' }));

app.get('/redirect', (req, res) => {
  console.log('GET redirected')
  res.status(200).send('redirected')
})

app.post('/redirect', (req, res) => {
  console.log(`POST redirected, body is ${req.body}`)
  res.status(200).send('redirected')
})

app.get('/get-get', (req, res) => {
  console.log('New GET (get-get) request received')
  res.redirect(301, 'http://127.0.0.1:8083/redirect')
})

app.post('/post-post', (req, res) => {
  console.log('New POST (post-post) request received')
  res.redirect(307, 'http://localhost:8083/redirect')
})

app.listen(port, () => {
  console.log(`App listening on port ${port}`)
})
```

3. Run the Go application from *Step 1* to trigger the output in the *Step 2*'s application.

Output:

```
$ node app.js
[...]
App listening on port 8083
New POST (post-post) request received
POST redirected, body is hola
```

The following listing enumerates all affected files.

Affected files:

- `go-daml/pkg/auth/keycloak.go`
- `go-daml/pkg/client/scanproxy.go`

The excerpt shown below is from the scan-proxy implementation. It is evident that the `httpClient` fails to guard against redirects and instead follows redirects to any provided address. The Keycloak implementation suffers from a similar issue.

Affected code (`scanproxy.go`):

```
func NewScanProxyClient(baseURL string, provider auth.TokenProvider)
*ScanProxyClient {
    logger := log.Logger.With().Str("component", "scan-proxy-
client").Logger()

    return &ScanProxyClient{
        baseURL:      baseURL,
        httpClient:    &http.Client{Timeout: 30 * time.Second},
        tokenProvider: provider,
        logger:        logger,
    }
}
[...]
func (s *ScanProxyClient) do(ctx context.Context, method, path string,
body, result interface{}) error {
    [...]
    req, err := http.NewRequestWithContext(ctx, method, s.baseURL+path,
reqBody)
    if err != nil {
        return fmt.Errorf("failed to create request: %w", err)
    }
    [...]
    resp, err := s.httpClient.Do(req)
    [...]
}
```

To mitigate this issue, Cure53 recommends adding a function denying all requests to internal/private endpoints. Said function should be offered as `CheckRedirect` field in the HTTP client.

NOD-01-008 WP2: Memory exhaustion DoS via unbounded streams (*Medium*)

During a source code review of the `go-daml` repository and the Amulet implementation, it was noticed that the `tokenstandard.go` file uses several streaming functions of the `ledger` API services. These streaming functions, such as `GetActiveContracts`, return a channel, thereby allowing the caller to consume all results from the channel until no further results are available.

Reviewing the handling of result channels in the *tokenStandardController* struct's functions and the *ContractQuery[T]* struct uncovered several memory exhaustion vulnerabilities. These issues are caused by the fact that the SDK attempts to keep all results in arrays, and hence in the application's memory.

In particular, a rogue upstream *ledger* node could return millions of entries one-by-one whenever the SDK queries for active contracts through the *GetActiveContracts* function. In this scenario, the SDK will add each of these results to an array. Accumulating an overwhelming number of results in an array will eventually bring the application using the SDK into an out-of-memory situation. A crash of the process would ensue.

The excerpt below demonstrates the issue for the *ListContractsByInterface* function. It uses the *GetActiveContracts* function of the state-service of the SDK. In the *for* loop, the *ListContractsByInterface* function grows the *contracts* array by continuously appending new entries, doing so as long as the channel returns new results.

Affected file #1:

go-daml/pkg/wallet/tokenstandard.go

Affected code #1:

```
func (t *tokenStandardController) ListContractsByInterface(ctx
context.Context, interfaceID string) ([]*model.CreatedEvent, error) {
    [...]
    stream, errChan := t.damlClient.StateService.GetActiveContracts(ctx,
req)

    var contracts []*model.CreatedEvent
    for {
        select {
        case resp, ok := <-stream:
            if !ok {
                return contracts, nil
            }
            if entry, ok := resp.ContractEntry.
(*model.ActiveContractEntry); ok {
                if entry.ActiveContract != nil &&
entry.ActiveContract.CreatedEvent != nil {
                    contracts = append(contracts,
entry.ActiveContract.CreatedEvent)
                }
            }
        }
        [...]
    }
}
```

The excerpt below demonstrates the issue for all functions of the *tokenstandard.go* file that use the functions of *query.go*, such as *FindContractsByTemplate*, for example. The functions in *query.go* rely on the *collect* function of *ContractQuery[T]* shown below. It needs to be noted that the function present here utilizes *GetActiveContracts*. This grows the *results* array until the *GetActiveContracts* function no longer pushes results into the channel.

Affected file #2:

go-daml/pkg/client/query.go

Affected code #2:

```
func (c *ContractQuery[T]) collect(ctx context.Context, query
contractQuery) ([]Contract[T], error) {
    var results []Contract[T]
    err := c.scanActiveContractsByTemplate(ctx, query, func(evt
activeContractEvent) (bool, error) {
        [...]
        results = append(results, Contract[T]{
            ContractID:      evt.contractID,
            TemplateID:      evt.templateID,
            CreatedAt:        evt.createdAt,
            CreatedEventBlob: evt.createdEventBlob,
            Data:             t,
        })
        return false, nil
    })
    [...]
func (c *ContractQuery[T]) scanActiveContractsByTemplate(
    ctx context.Context,
    query contractQuery,
    onEvent func(evt activeContractEvent) (stop bool, err error),
) error {
    [...]
    respCh, errCh := c.cl.StateService.GetActiveContracts(streamCtx, req)
    for {
        select {
            case resp, ok := <-respCh:
                [...]
                stop, err := onEvent(activeContractEvent{
                    contractID:      evt.ContractID,
                    templateID:      evt.TemplateID,
                    arguments:        arguments,
                    createdAt:        evt.CreatedAt,
                    createdEventBlob: evt.CreatedEventBlob,
                })
                [...]
            }
        }
    }
```

To mitigate this issue Cure53 advises equipping all publicly exposed SDK functions relying on such streaming responses with an *offset* and *limit* parameter. What is more, Noders should adopt a paginated data access and enforce a safe upper-bound value on the *limit* parameter.

NOD-01-010 WP1: Improper handling of TLS config parameters (*Medium*)

The SDK establishes two gRPC connections, namely a *ledger* and an *admin* connection, to the Canton network. As part of the configuration of these connections, the SDK enables developers to specify some values for setting up TLS. The developers set these TLS configurations by either using the *NewClient* function the *WithTLSConfig* function or by setting the *Client* struct's or *DamClient* struct's *config* field directly.

Along with these values, the developers can provide a server name and set the *InsecureSkipVerify* boolean. The *tls.Config* struct of the official *crypto/tls* package, ultimately used to create the gRPC connections, supports many more configuration options. Among them are selecting cipher-suites, TLS min/max versions and renegotiation.

However, the SDK fails to give developers the option to use these configuration options, and hence insufficiently supports the secure configuration of the *ledger* and *admin* gRPC connections. Furthermore, the *CertFile* property of the *Config* struct, even though it is checked for non-emptiness, is not set on the utilized *tls.Config* instance when the *Client* struct is used for the SDK.

Cure53 also noticed that the *WithTLSConfig* function of the *DamClient* struct is not setting the provided *cfg* parameter directly. Instead, it only sets the *CertFile* property of the utilized configuration, without setting the *ServerName* and *InsecureSkipVerify* fields. Because the *DamClient* struct uses the *NewClient* function together with its *Connect* function, none of the supported configuration values for TLS can be used for the *DamClient* struct. Resultantly, the TLS connections will neither check the server's name nor use the selected certificate file.

Due to the fact that these parameters are effectively not applied by the SDK even though exposed, it was decided to file this issue as a vulnerability. It totally breaks the expectations of a developer using the SDK together with TLS parameters, as the developer expects that the server's name is set, and the supplied certificate file is being used by the underlying connections.

The current behaviors put the connections to both *ledger* and *admin* API at risk of insecure configurations and settings. Specifically, the developers cannot enforce minimum TLS versions, opt-out from unsafe cipher-suites, or implement pub-key or certificate pinning if necessary. Not being able to enforce such settings could aid malicious actors in attempts to attack the TLS connections established by the SDK.

The excerpt below demonstrates the issue. It is clear that the *buildTLSConfig* function creates an instance of *tls.Config* and sets only the values of *ServerName* and *InsecureSkipVerify*. Furthermore, the *CertFile* option is checked for non-emptiness, but never set on *tlsConfig*.

Affected file #1:

go-daml/pkg/client/client.go

Affected code #1:

```
func (c *Client) buildTLSConfig() *tls.Config {
    tlsConfig := &tls.Config{
        ServerName:      c.config.TLS.ServerName,
        InsecureSkipVerify: c.config.TLS.InsecureSkipVerify,
    }

    if c.config.TLS.CertFile != "" {
    }

    return tlsConfig
}
```

The excerpt below highlights the only value that is set in the TLS configuration when using the *WithTLSConfig* function of the *DamlClient* struct.

Affected file #2:

go-daml/pkg/client/daml.go

Affected code #2:

```
func (c *DamlClient) WithTLSConfig(cfg TlsConfig) *DamlClient {
    c.config.TLS = &TLSConfig{
        CertFile: cfg.Certificate,
    }
    return c
}
```

To mitigate this issue Cure53 advises to use an instance of the *tls.Config* struct in the configuration of the *Client* and *DamlClient* structs. This should be done in place of using another struct mirroring the *tls.Config* struct from the official Go package and copying its values. The optimal solution would give developers the flexibility to configure any settings supported by the *tls.Config* struct¹.

¹ <https://pkg.go.dev/crypto/tls#Config>

NOD-01-012 WP1: Float64 precision loss in JSON unmarshalling (*Medium*)

While analyzing the different JSON codec functions, a precision loss vulnerability was found in the SDK (`pkg/codec/json_codec.go`). The `Unmarshal` function at line 76 uses Go's standard `json.Unmarshal()` to decode JSON data into an `interface{}` type. This causes all JSON numbers to be converted to IEEE 754 float64 values before the SDK processes them further. Since float64 cannot exactly represent most decimal fractions, precision is silently lost at this stage.

The imprecise float64 value then flows through `fromDynamicValue` → `assignValue` → `assignBigIntValue`, where it is scaled by 10^{10} to produce the internal `NUMERIC` representation. For example, the JSON value 5.1 becomes float64(5.09999999999999964...), ultimately producing 50999999999 instead of the correct 51000000000.

A developer using this vulnerable function to parse monetary amounts from JSON sources where numeric values are represented as unquoted JSON numbers (e.g., API responses, configuration files, or user input) would silently receive imprecise values. This violates the precision guarantees expected for financial calculations and could lead to incorrect transfer amounts, failed balance reconciliations, or cumulative rounding discrepancies across transaction processing.

The following PoC reproduces the issue by using the SDK's `Unmarshal` function, which internally converts the JSON input to float64 before passing it to `assignBigIntValue` and printing the conversion result. Running the application demonstrates the observed flaw.

PoC application:

```
package main

import (
    "fmt"
    "math/big"

    "github.com/noders-team/go-daml/pkg/codec"
    "github.com/noders-team/go-daml/pkg/types"
)

func main() {
    c := codec.NewJsonCodec()

    var result types.NUMERIC
    _ = c.Unmarshal([]byte(`5.1`), &result)

    actual := (*big.Int)(result)
```

```
expected := big.NewInt(51000000000)

fmt.Printf("Input:    5.1 (JSON number, unquoted)\n")
fmt.Printf("Expected: %s \n", expected)
fmt.Printf("Actual:    %s \n", actual)
}
```

PoC output:

```
Input:    5.1 (JSON number, unquoted)
Expected: 51000000000
Actual:   50999999999
```

Affected file:

pkg/codec/json_codec.go

Affected code:

```
[...]
// Unmarshal converts JSON bytes back to a DAML structure following
transcode patterns
func (codec *JsonCodec) Unmarshal(data []byte, target interface{}) error {
    var intermediate interface{}
    if err := json.Unmarshal(data, &intermediate); err != nil {
        return fmt.Errorf("failed to unmarshal JSON: %w", err)
    }

    return codec.fromDynamicValue(intermediate, target)
}
[...]
```

Cure53 recommends replacing *json.Unmarshal()* with *json.NewDecoder()* configured with *UseNumber()*. This preserves JSON numbers as strings, preventing the loss of precision in the float64 conversions. Numeric values would then reach *assignBigIntValue* as strings. At that location exact parsing via *big.Rat.SetString* is already implemented.

NOD-01-013 WP1: Inconsistency in BigInt scaling of JSON codec (*Medium*)

The *assignBigIntValue* function in *pkg/codec/json_codec.go* applies different scaling logic depending on the format of the input value. When a *NUMERIC* value is provided as a string containing a decimal point (e.g., 5.0), the function correctly scales it by 10^{10} via *big.Rat* to produce the internal representation. However, when the same semantic value is provided as a string without a decimal point (e.g., 5), it is parsed directly via *big.Int.SetString* without any scaling applied.

This causes semantically identical values to produce different internal representations. The string `5.0` correctly becomes `big.Int(50000000000)`, while the string `5` becomes `big.Int(5)` - a difference of factor 10,000,000,000. The same inconsistency applies to JSON integers, where the value `5` is treated identically to the string `5` and bypasses scaling entirely.

A developer using the SDK's `Unmarshal` function to parse monetary amounts from JSON sources where values lack an explicit decimal point would silently receive values that are 10^{10} times smaller than intended. This could lead to transfers of negligible amounts instead of the intended values. To that end, the problem could translate to effective loss of funds from the sender's perspective, or undetected underpayments in automated transaction processing.

The following PoC reproduces the issue by passing different representations of the value `5` through the SDK's `Unmarshal` function, which internally invokes `assignBigIntValue`. Running the application demonstrates that only the string with a decimal point is correctly scaled; all other input formats produce a value that is off by a factor of 10^{10} .

PoC application:

```
package main

import (
    "fmt"
    "math/big"

    "github.com/noders-team/go-daml/pkg/codec"
    "github.com/noders-team/go-daml/pkg/types"
)

func main() {
    c := codec.NewJsonCodec()

    inputs := []struct {
        name string
        json string
    }{
        {"case string (integer)", `"5"`},
        {"case string (decimal)", `"5.0"`},
        {"case float64 (integer)", `5`},
        {"case float64 (non-int)", `5.1`},
    }

    for _, input := range inputs {
        var result types.NUMERIC

        _ = c.Unmarshal([]byte(input.json), &result)
```

```
        fmt.Printf("%-30s → big.Int = %s\n", input.name+"  
("+input.json+")", (*big.Int)(result))  
    }  
}
```

PoC output:

```
case string (integer) ("5")      → big.Int = 5  
case string (decimal) ("5.0")   → big.Int = 50000000000  
case float64 (integer) (5)      → big.Int = 5  
case float64 (non-int) (5.1)    → big.Int = 50999999999
```

Affected file:

pkg/codec/json_codec.go

Affected code:

```
[...]  
func (codec *JsonCodec) assignBigIntValue(jsonValue interface{}, target  
reflect.Value, typeName string, converter func(*big.Int) reflect.Value)  
error {  
    switch v := jsonValue.(type) {  
    case string:  
        if bi, ok := new(big.Int).SetString(v, 10); ok {  
            target.Set(converter(bi))  
            return nil  
        }  
        if rat, ok := new(big.Rat).SetString(v); ok {  
            scaledInt := new(big.Int)  
            scaledInt.Mul(rat.Num(), big.NewInt(100000000000))  
            scaledInt.Div(scaledInt, rat.Denom())  
            target.Set(converter(scaledInt))  
            return nil  
        }  
        return fmt.Errorf("invalid string format for %s: %s",  
typeName, v)  
    case float64:  
        if v == float64(int64(v)) {  
            bi := big.NewInt(int64(v))  
            target.Set(converter(bi))  
            return nil  
        }  
        rat := new(big.Rat).SetFloat64(v)  
        scaledInt := new(big.Int)  
        scaledInt.Mul(rat.Num(), big.NewInt(100000000000))  
        scaledInt.Div(scaledInt, rat.Denom())  
        target.Set(converter(scaledInt))  
        return nil  
    case int64:
```

```
        bi := big.NewInt(v)
        target.Set(converter(bi))
        return nil
    default:
        return fmt.Errorf("expected string or number for %s, got %T",
            typeName, jsonValue)
    }
}
```

Cure53 recommends applying consistent scaling across all branches. Every input - whether it is a string, integer, or float - that represents a *NUMERIC* value should be uniformly scaled by 10^{10} before being stored as *big.Int*. Specifically, the *big.Int.SetString* branch (line 670) and the *int64* branch (line 694) should route their values through the same *big.Rat*-based scaling logic as the one already implemented for decimal values.

Additionally, the support for float64 inputs should be reevaluated, as it introduces a separate precision loss issue (see [NOD-01-012](#)). The Noders team needs to ensure that the encoding and decoding paths produce consistent results that preserve correct round-trip behaviors.

Miscellaneous Issues

This section covers any and all noteworthy findings that did not incur an exploit but may assist an attacker in successfully achieving malicious objectives in the future. Most of these results are vulnerable code snippets that did not provide an easy method by which to be called. Conclusively, while a vulnerability is present, an exploit may not always be possible.

NOD-01-001 WP1: Missing support for mTLS in SDK (*Low*)

During the source code audit, Cure53 identified that the SDK currently lacks support for mutual TLS (mTLS) authentication. While the Canton nodes provide native support for mTLS to secure API communications², this capability is not exposed or supported within the analyzed SDK. Notably, utilizing mTLS would make stealing *auth* tokens impossible, as no secrets need to be transmitted over the network.

To bolster the overall security posture of the application, Cure53 recommends evaluating the feasibility of implementing mTLS support within the SDK. Enabling this feature would allow the SDK to fully leverage the security mechanisms already provided by the Canton nodes, ensuring that both the client and the server cryptographically verify each other's identities before establishing a connection.

NOD-01-002 WP2: *Float64* precision issue in unreferenced functions (*Info*)

Several functions unreferenced throughout the codebase and not reachable at runtime could be identified during the assessment. These include *validateWeight*, *beneficiariesToArgs*, *unwrapExercise*, *wrapDelegateProxy*, and *FetchPendingAllocationView* in *pkg/wallet/tokenstandard.go*.

Among these, *validateWeight* contains a latent precision defect. It sums up beneficiary weights using *float64* addition and compares the result against 1.0. Due to IEEE 754 floating-point representation, certain valid decimal sums can produce rounding errors (e.g., $0.1 + 0.2 + 0.3 + 0.4$ evaluates to 0.9999... rather than 1.0), which could cause legitimate weight distributions to pass validation when they should be rejected, or vice versa.

Even if this function became reachable, the practical impact would remain limited. Fortunately, *validateWeight* performs a client-side pre-validation before the data is submitted to the *ledger*, which enforces its own authoritative validation. A rounding error in the client-side check would therefore result in either a redundant rejection or a submission that the *ledger* validates independently.

² <https://docs.daml.com/canton/usermanual/apis.html>

Affected file:

pkg/wallet/tokenstandard.go

Affected code:

```
[...]
func (t *tokenStandardController) validateWeight(bs []*model.Beneficiary)
error {
    var sum float64
    for _, b := range bs {
        sum += b.Weight
    }
    if sum > 1.0 {
        return fmt.Errorf("sum of beneficiary weights is larger than
1")
    }
    return nil
}
[...]
```

Cure53 recommends removing unreferenced functions to maintain a clean codebase, as code hygiene is directly tied to security. Orphaned code is more likely to be overlooked, creating a risk of latent vulnerabilities being introduced if the code is later referenced without proper review. If these functions are actually intended for future use, Cure53 advises replacing *float64* with *decimal.Decimal* for weight arithmetic to eliminate the precision risk before activation.

NOD-01-004 WP1: Unbound recursion in response decoding ([Info](#))

It was discovered that the JSON encoder/decoder of the SDK adopts recursion without restricting its nesting depth. Each function call consumes stack memory, hence such unbounded recursions often lead to stack overflow/out-of-memory situations if not constrained.

In this particular case, exploitability and impact remain negligible because the SDK first parses untrusted data using *json.Unmarshal*, guarding the project against deep object graphs. However, it was decided to still report this issue due to the fact that this issue could become exploitable in the future. Specifically, risk could emerge if either the *json.Unmarshal* function changes, or the affected function is exposed directly by the SDK.

Upon further review, it was discovered that other code parts suffer from the same issue. Specifically, the *valueFromProto* function in the *converters.go* for the *ledger* services contains an unconstrained recursion for *optionals*, records and other composite data structures.

The excerpt below shows the recursion for the *struct* type. It is evident that the *assignValue* function calls the *assignStructValue* function that in turn calls again the *assignValue* function.

Affected file #1:

go-daml/pkg/codec/json_codec.go

Affected code #1:

```
func (codec *JsonCodec) assignValue(jsonValue interface{}, target
reflect.Value) error {
    [...]
    targetType := target.Type()

    switch targetType {
    [...]
    if target.Kind() == reflect.Struct {
        return codec.assignStructValue(jsonValue, target)
    }
    [...]
    }
}

[...]
func (codec *JsonCodec) assignStructValue(jsonValue interface{}, target
reflect.Value) error {
    if m, ok := jsonValue.(map[string]interface{}); ok {
        [...]
        for i := 0; i < target.NumField(); i++ {
            [...]
            if jsonVal, exists := m[fieldName]; exists {
                if err := codec.assignValue(jsonVal, field);
err != nil {
                    return fmt.Errorf("failed to assign field
%s: %w", fieldName, err)
                }
            }
        }

        return nil
    }
    return fmt.Errorf("expected object for struct, got %T", jsonValue)
}
```

The following list enumerates other types adopting recursion for decoding:

- *TypedMap*
- *Reltime*
- *Tuple2*

- *Tuple3*
- *Slice*
- *Struct*

The excerpt below demonstrates the issue for the *valueFromProto* function. It is clear that the function calls itself recursively for compound data types without any constraint on the recursion depth.

Affected file #2:

go-daml/pkg/service/ledger/converters.go

Affected code #2:

```
func valueFromProto(pb *v2.Value) interface{} {  
    [...]  
    switch v := pb.Sum.(type) {  
    [...]  
    case *v2.Value_Optional:  
        if v.Optional.Value != nil {  
            return valueFromProto(v.Optional.Value)  
        }  
        return nil  
    case *v2.Value_List:  
        result := make([]interface{}, len(v.List.Elements))  
        for i, elem := range v.List.Elements {  
            result[i] = valueFromProto(elem)  
        }  
        return result  
    case *v2.Value_Record:  
        [...]  
        record := make(map[string]interface{})  
        for _, field := range v.Record.Fields {  
            record[field.Label] = valueFromProto(field.Value)  
        }  
        return record  
    case *v2.Value_TextMap:  
        [...]  
        result := make(map[string]interface{})  
        for _, entry := range v.TextMap.Entries {  
            result[entry.Key] = valueFromProto(entry.Value)  
        }  
        return result  
    [...]  
    case *v2.Value_Variant:  
        if v.Variant != nil {  
            return map[string]interface{}{  
                "tag": v.Variant.Constructor,  

```

```

        "value": valueFromProto(v.Variant.Value),
    }
}
return nil
default:
    return nil
}
}

```

To mitigate this issue Cure53 advises implementing a guard against deep recursions. For that purpose, decoding/parsing functions should normally increment a counter when entering a new recursion level. They need to return from the recursion with an error in case the counter exceeds a predefined threshold.

NOD-01-007 WP1: Potential infinite loop in *ledger* API services (*Info*)

The SDK implements individual clients/services for different kinds of APIs/nodes of a Canton network. The SDK has dedicated services for the *admin* API but also the *ledger* API. For example, as part of the *ledger* API services, the SDK contains an implementation handling command completion. This implementation reads from a stream until either an error is returned, or a non-empty response is received.

However, in certain cases it is possible for the *ledger* API to return a *nil* response and no error, effectively resulting in an infinite loop. Further review of the SDK revealed that also the function for reading the active contracts, namely the *GetActiveContracts* function, and the *GetUpdates* function, suffer from the same issue.

Discussing this issue with the Noders team revealed that this situation can only occur in case a *nodes* API is changed without the SDK getting updated to that contract. Due to the rare nature of this situation, and the fact that an attacker requires full control over a node to exploit this issue, it was assigned an *Info* severity level.

Nevertheless, the issue is demonstrated in the excerpt below. It is evident that the *CompletionStream* function continues reading from the *stream.Recv()* function until either an error is returned or a *non-nil* response is decoded.

Affected file #1:

go-daml/pkg/service/ledger/command_completion.go

Affected code #1:

```

func (c *commandCompletion) CompletionStream(ctx context.Context, req
*model.CompletionStreamRequest) (<-chan *model.CompletionStreamResponse, <-
chan error) {
    [...]

```

```

stream, err := c.client.CompletionStream(ctx, streamReq)
if err != nil {
    errCh := make(chan error, 1)
    errCh <- err
    close(errCh)
    return nil, errCh
}
[...]
go func() {
    defer close(responseCh)
    defer close(errCh)

    for {
        resp, err := stream.Recv()
        if err == io.EOF {
            return
        }
        if err != nil {
            errCh <- err
            return
        }

        modelResp := completionStreamResponseFromProto(resp)
        if modelResp != nil {
            select {
            case responseCh <- modelResp:
            case <-ctx.Done():
                return
            }
        }
    }
}()

return responseCh, errCh
}

func completionStreamResponseFromProto(pb *v2.CompletionStreamResponse)
*model.CompletionStreamResponse {
    if pb == nil {
        return nil
    }
    [...]
}

```

The excerpt below demonstrates the issue for the *GetActiveContracts* function. Also here the function continues with the *for* loop until either an error is read or a *non-nil* response is decoded.

Affected file #2:

go-daml/pkg/service/ledger/state_service.go

Affected code #2:

```
func (c *stateService) GetActiveContracts(ctx context.Context, req
*model.GetActiveContractsRequest) (<-chan
*model.GetActiveContractsResponse, <-chan error) {
    [...]
    stream, err := c.client.GetActiveContracts(ctx, protoReq)
    if err != nil {
        errCh := make(chan error, 1)
        errCh <- err
        close(errCh)
        return nil, errCh
    }
    [...]
    go func() {
        defer close(responseCh)
        defer close(errCh)

        for {
            resp, err := stream.Recv()
            if err == io.EOF {
                return
            }
            if err != nil {
                errCh <- err
                return
            }

            modelResp := getActiveContractsResponseFromProto(resp)
            if modelResp != nil {
                select {
                    case responseCh <- modelResp:
                    case <-ctx.Done():
                        return
                }
            }
        }
    }()

    return responseCh, errCh
}
[...]
func getActiveContractsResponseFromProto(pb *v2.GetActiveContractsResponse)
*model.GetActiveContractsResponse {
```

```
        if pb == nil {  
            return nil  
        }  
        [...]  
    }  
}
```

The excerpt below demonstrates the behavior of the *GetUpdates* function. The function stays in the *for* loop until either an error is returned or a *non-nil* response is decoded.

Affected file #3:

go-daml/pkg/service/ledger/update_service.go

Affected code #3:

```
func (c *updateService) GetUpdates(ctx context.Context, req  
*model.GetUpdatesRequest) (<-chan *model.GetUpdatesResponse, <-chan error)  
{  
    [...]  
    stream, err := c.client.GetUpdates(ctx, protoReq)  
    [...]  
    go func() {  
        defer close(responseCh)  
        defer close(errCh)  
  
        for {  
            resp, err := stream.Recv()  
            if err == io.EOF {  
                return  
            }  
            if err != nil {  
                errCh <- err  
                return  
            }  
  
            modelResp := getUpdatesResponseFromProto(resp)  
            if modelResp != nil {  
                select {  
                case responseCh <- modelResp:  
                case <-ctx.Done():  
                    return  
                }  
            }  
        }  
    }()  
  
    return responseCh, errCh  
}  
[...]
```

```
func getUpdatesResponseFromProto (pb *v2.GetUpdatesResponse)
*model.GetUpdatesResponse {
    if pb == nil {
        return nil
    }
    [...]
}
```

To mitigate this issue, Cure53 advises returning an error in case a *nil* response is returned from the *ledger* API. This behavior is an indication of a contract version mismatch between *ledger* API and SDK.

NOD-01-009 WP2: Data unreadability due to no query offset ([Info](#))

The *tokenstandard.go* file contains several functions that query for contracts recorded on the *ledger*. To that end, these functions utilize the *FindContractsByInterface* function. This function returns a complete list of all contracts matching the search criteria specified, which in itself could be used by a rogue *ledger* node to mount a DoS (see [NOD-01-008](#)).

Using the complete list, the functions affected by this issue copy the elements of the result list of the *FindContractsByInterface* function into a result array. To restrict the size of this result array, most of these functions require a *limit* parameter. However, it was found that the exposed functions do not support an *offset* parameter. This can result in situations where users of the SDK cannot query for some items, even with honest *ledger* nodes.

Suppose the *FindContractsByInterface* function returns 700MB results, but the application process using the SDK can only handle up to 1GB of memory. In this situation, the application will run out of memory for all *limit* values larger than 300MB. This is due to the fact that the exposed functions copy the query results into a result array, and hence the results of *FindContractsByInterface* together with the result array are constrained by 1GB.

Particularly, attempting to access items located in memory locations above 300MB would exceed the process memory, resulting in a crash of the application using the SDK. Even though this issue does not have a direct security impact, it was confirmed by the customer that this flaw should be filed in the report as part of this engagement.

The excerpt below demonstrates the issue in the *ListHoldingUtxos* function. It is clear that the function does not support an *offset* parameter. It first creates the *result* array, and then consequently loops over all elements in the *contracts* array. As a consequence, both arrays are kept in memory simultaneously. It must be noted that the *tokenstandard.go* file contains several functions using the same pattern.

Affected file:

go-daml/pkg/wallet/tokenstandard.go

Affected code:

```
func (t *tokenStandardController) ListHoldingUtxos(ctx context.Context,
includeLocked bool, limit int) ([]*model.HoldingUTXO, error) {
    [...]
    now := time.Now()
    result := make([]*model.HoldingUTXO, 0, len(contracts))
    for _, c := range contracts {
        if !includeLocked && t.isHoldingLocked(c.Data, now) {
            continue
        }
        result = append(result, &model.HoldingUTXO{
            ContractID:      c.ContractID,
            Amount:          t.numericToDecimal(c.Data.Amount),
            InstrumentID:     string(c.Data.InstrumentId.Id),
            InstrumentAdmin: string(c.Data.InstrumentId.Admin),
            Owner:           string(c.Data.Owner),
            Lock:            t.lockToMap(c.Data.Lock),
            CreatedEventBlob: c.CreatedEventBlob,
        })
        if limit > 0 && len(result) >= limit {
            break
        }
    }

    t.logger.Debug().
        Str("partyID", string(partyID)).
        Int("holdings", len(result)).
        Msg("listed holding utxos via Holding interface query")

    return result, nil
}
```

Cure53 recommends adding an optional *offset* parameter to all functions which query data and only add items to the corresponding result array starting from that *offset*.

NOD-01-011 WP1: *Ledger* and *admin* gRPC connections use same config (*Low*)

The SDK creates two connections to nodes of the Canton network, namely a *ledger* and an optional *admin* gRPC connection. The *Client* struct of the SDK calls the *buildDialOptions* function to generate the dial options prior to establishing any of the connections. The observed dial options include the TLS configuration but also set the authentication provider, either via static *bearer* token or Keycloak.

It was discovered that the SDK forces the developer to use the same dial options on both connections. This puts connections at risk in case one connection is using less secure dial options than the other. If both connections require *bearer* tokens, the connection must use the same token for authentication. This implies a breach of one connection that will impact security of the other connection.

The issue is demonstrated in the excerpt below. Both connections use the *opts* variable for creating the gRPC client via *grpc.NewClient*.

Affected file:

go-daml/pkg/client/client.go

Affected code:

```
func (c *Client) Connect(ctx context.Context) (*Connection, error) {  
    opts := c.buildDialOptions()  
  
    conn, err := grpc.NewClient(c.config.Address, opts...)  
    [...]  
    c.conn = conn  
  
    var adminConn *grpc.ClientConn  
    if c.config.AdminAddress != "" {  
        adminConn, err = grpc.NewClient(c.config.AdminAddress,  
    opts...)  
        [...]  
        c.adminConn = adminConn  
    }  
  
    return NewConnection(c, conn, adminConn), nil  
}
```

It is recommended to differentiate between *ledger* and *admin* connection configuration options. For that purpose, the *NewClient* function should accept a *ledgerConfiguration* and *adminConfiguration* parameter, both of type *Config*, and create the respective dial options for the connections when establishing them.

NOD-01-014 WP1: No length check in topology-manager could lead to panic ([Info](#))

During a source code review of the *go-daml* repository, it was noticed that the SDK implements two wrappers for the topology-management in a Canton network. The writer for topology transactions offers the *DropTemporaryTopologyStore* function through its API, but this function could potentially cause panic due to out-of-bounds string access.

In particular, the function takes the *StoreID* field of the *req* parameter and attempts to slice its value starting from index 10 within the string. If the string is shorter than 10 characters, the SDK will raise a panic, which in turn causes an application crash if not handled within a *recover* function.

Depending on the application using the SDK and whether the application checks the string for a minimum length of 10, this could be used for DoS attacks. The excerpt below shows the definition of the *StoreID* struct. Quite clearly, this comprises a single string field with name *Value*.

Affected file #1:

go-daml/pkg/model/topology.go

Affected code #1:

```
type StoreID struct {  
    Value string  
}  
[...]  
type DropTemporaryTopologyStoreRequest struct {  
    StoreID *StoreID  
}
```

The excerpt below highlights the unguarded slicing. The function clearly slices the *req.StoreID.Value* field without checking its length.

Affected file #2:

go-daml/pkg/service/topology/topology_manager_write.go

Affected code #2:

```
func (c *topologyManagerWrite) DropTemporaryTopologyStore(ctx  
context.Context, req *model.DropTemporaryTopologyStoreRequest)  
(*model.DropTemporaryTopologyStoreResponse, error) {  
    protoReq := &topov30.DropTemporaryTopologyStoreRequest{  
        StoreId: &topov30.StoreId_Temporary{  
            Name: req.StoreID.Value[10:],  
        },  
    }  
  
    _, err := c.client.DropTemporaryTopologyStore(ctx, protoReq)  
    if err != nil {  
        return nil, err  
    }  
  
    return &model.DropTemporaryTopologyStoreResponse{}, nil  
}
```

To mitigate this issue Cure53 advises implementing rigorous input data validations for all input parameters of the SDK as part of the defense-in-depth measures. This would include checks of the structure, expected format and length of input fields, amongst other additional defenses.

NOD-01-015 WP1: Import of vulnerable packages and dependencies ([Info](#))

An assessment of the dependencies of the SDK uncovered numerous modules with publicly known vulnerabilities and security issues. Relying on outdated and vulnerable dependencies is highly discouraged from a security perspective. Even though those vulnerabilities may not be directly reachable for an attacker at the moment, or the SDK does not currently use a vulnerable function, it unnecessarily increases the attack surface on the SDK.

In particular, as the SDK evolves over time, it may start using vulnerable functions from the imported packages. As a consequence, these vulnerabilities could become exploitable in the future. Notably, this issue was uncovered by using the *govulncheck* tool³. The output shown below illustrates that the SDK suffers from numerous vulnerabilities in various imported modules/packages.

Output:

```
$ govulncheck ./...  
=== Symbol Results ===
```

```
Vulnerability #1: GO-2026-6061  
[...]
```

```
Your code is affected by 28 vulnerabilities from 4 modules and the Go  
standard library.
```

```
This scan also found 19 vulnerabilities in packages you import and 6  
vulnerabilities in modules you require, but your code doesn't appear to  
call
```

```
these vulnerabilities.
```

```
Use '-show verbose' for more details.
```

Cure53 recommends adopting a better-secured development process. As part of this process, the development team of the SDK should review and scan all dependencies for publicly known vulnerabilities on a regular basis. Vulnerable packages/modules should accordingly be updated right away.

³ <https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck>

Conclusions

The goal of this *NOD-01* assessment was for the Cure53 to assess the security of the *godami* SDK and its Wallet & Amulet standard implementation. To address it, the inspection was carried out by the testing team in August 2026 over two structured work packages. WP1 entailed audits & reviews against the core SDK, whereas WP2 focused on the Wallet & Amulet standard implementation.

All necessary information to conduct the audit was provided prior to the start of this *NOD-01* project. This included a link to the GitHub repository containing the source code of the SDK, and access to a Canton test environment. The repository contains valuable documentation to better understand the purpose and intended use-cases of the SDK.

Over the course of the project, which initiates the cooperation between Cure53 and Noders, the teams shared a dedicated Slack channel to facilitate all communications. Help was provided by the Noders developer whenever requested, while the Cure53 consultants not only gave frequent status updates but also live-reported all findings in the Markdown format to facilitate the process of fix development and deployment.

The SDK in scope corresponds to a software development kit for Canton networks. It implements connection handling and their setup in terms of gRPC connections (including support for TLS). It is integrated with two authentication providers, namely static JWTs or Keycloak. To facilitate upstream communication, the SDK offers several services that wrap the underlying low-level APIs to Canton. As the SDK also supports Wallet & Amulet flows, it further contains implementation logic for the flows deployed in the corresponding APIs.

The assessment of the SDK focused primarily on transaction processing mechanisms and signing, cryptographic implementations, authentication routines, and data handling workflows. The flows of the Wallet & Amulet standards have also been scrutinized with special attention paid to conversion issues which could result in either an illicit supplementation or loss of funds. Evaluation was conducted through targeted static source code analysis, tracing logic paths across core modules.

To complement code-level reviews, automated scanning using Gitleaks and TruffleHog was performed across the repository's Git history. This revealed no leaked secrets, hardcoded credentials, or exposed sensitive parameters. As such, it appears that the development team at Noders takes care to keep sensitive information out of the repository.

The assessment of cryptographic primitives and key management routines revealed a generally safe reliance on standard Go libraries (such as *crypto/ed25519* and *crypto/sha256*) rather than custom implementations. Ed25519 keys are securely generated using secure system randomness (*rand.Reader* from *crypto/rand*) do not leak. Signature verification within *VerifySignedTxHash* safely defers to Go's internal *ed25519.Verify*.

On a minor note, many functions fail to perform input data validation checks on parameters where certain formats are expected. For example, the *VerifySignedTxHash* function has the *hashBase64* parameter which is expected to be a base64-encoded hash. However, the function fails to check the length of the decoded string. It also does not inspect whether the decoded string contains hex characters exclusively. The same observation applies to the *SignTransactionHash* function. Due to the fact no security impact was identified, it was decided to not report this as separate issues in this report.

Cure53 scrutinized the codebase for any use of sinks that could facilitate or result in code execution issues. It was positively noted that the SDK does not make direct use of any of such sinks, which results in good security outcomes. At the same time, the SDK interacts with several different upstream services and APIs, and hence, Cure53 scrutinized the code base for attacks that could be mounted by a rogue upstream service.

One of the investigated vectors entails Denial-of-Service attacks via malicious responses. While analyzing the scan-proxy implementation, it became apparent that incoming HTTP responses' body sizes are not explicitly restricted from upstream servers. The vulnerable implementation is even easier to exploit because GZip content is accepted by default.

From this perspective, an attacker can deliver highly compressed payloads (such as ZIP bombs) that exhaust system resources upon expansion. This presents a potential DoS vector, as outlined in [NOD-01-003](#). What is more, an upstream rogue *ledger* node could flood the SDK with responses in return streams, for example via the *GetActiveContracts* function. Due to the fact that the SDK attempts to keep all responses in memory, this results in memory exhaustion DoS, as described in [NOD-01-008](#). Lastly, the SDK also suffers from an infinite loop that could be triggered by an upstream node, as documented in [NOD-01-007](#).

A further attack stemming from rogue upstream nodes pertains to request forgeries. Particularly, a rogue upstream node could respond with HTTP redirection codes such as 301 or 307 to HTTP requests from the SDK. It was confirmed that the *http.Client* in use by the SDK follows, in its present configuration, such redirects to internal or private IP addresses. This results in a request forgery vulnerability ([NOD-01-006](#)), making it possible for a rogue upstream node to attack the infrastructure running in the internal network of the application using the SDK.

The authentication providers were also reviewed for any kind of flaws. As explained, the SDK supports two authentication providers, namely static JWTs and Keycloak. Cure53 wanted to find ways to redirect the secrets of the providers to a malicious server or to extract them by other ways, but no issues were identified in this area.

Next, focus has shifted to the TLS configuration and connection setups, with numerous issues identified. An analysis of the TLS configurations revealed that mutual TLS (mTLS) is currently not supported by the SDK, despite the underlying Canton node implementation allowing for mTLS authentication. While the absence of mTLS does not constitute a direct vulnerability, implementing this feature would significantly bolster the overall security posture of the SDK, as detailed in [NOD-01-001](#).

More surprisingly, the SDK supports only two parameters of the *tls.Config* struct of the *crypto/tls* package of Go, namely the *ServerName* and *InsecureSkipVerify* parameters. This heavily restricts the security configuration options of TLS connections to the *ledger* and *admin* API. When using the *DamlClient's WithTLSConfig* function, intended to set the TLS configuration, no TLS settings are applied by the SDK. These circumstances are summarized in [NOD-01-010](#).

In the same realm, it was also discovered that the *ledger* and the *admin* gRPC connections share the same dial options and authentication provider. This implies that a compromise of one connection equals compromise of the other one ([NOD-01-011](#)).

Given that the primary focus of the assessment was on identifying potential precision loss during number conversions, corresponding functionalities were examined in detail. This review identified at least two instances of improper implementation.

The first instance involves a number conversion function that is no longer referenced within the source code, as outlined in [NOD-01-002](#). The identification of these unreferenced conversion functions highlights a broader issue within the codebase. During the audit, several other unused functions were discovered, suggesting a systemic shortcoming in the development lifecycle regarding code hygiene.

A second and more critical area involves insecure parsing of JSON input, which relies on *float64* representations and can lead to the generation of faulty numerical values, as described in [NOD-01-012](#). Because improper processing could have a drastic impact depending on how a developer integrates the vulnerable function via the SDK, it is highly recommended that the Noders development team carefully review and harden these implementations to ensure strict numerical accuracy.

Additionally, the analysis revealed an inconsistency in the *NUMERIC-to-BigInt* conversion logic. The *assignBigIntValue* function applies decimal scaling selectively based on input format, causing 5 and 5.0 to produce internal representations that differ by a factor of 10,000,000,000. This contradicts the official *DAML-LF* JSON Encoding specification, which treats both formats equivalently, as outlined in [NOD-01-013](#).

Besides vulnerabilities emerging from conversion faults, it was also investigated if the decoding functions suffer from other issues - such as potential DoS situations or insecure deserializations. It was found that the response decoding relies on recursions which could potentially result in stack overflow situations ([NOD-01-004](#)).

Issues emerging from wrong usage of the SDK, or insufficient input data validations before processing data, were also in scope. Corresponding research led to the discovery of a path traversal vulnerability that results in requests to unintended upstream paths ([NOD-01-005](#)). Particularly, in case the application using the SDK fails to check for traversal payloads, attackers could reach unintended endpoints through the SDK.

Another issue arises from insufficient length checks on an input parameter of a function of the topology-manager's *write* functionality. Due to this missing check, the SDK could raise a panic which, if not handled by a *recover* handler in the application, crashes the process ([NOD-01-014](#)). A bug without security implications was also reported to highlight the lack of *offset* parameter in queries ([NOD-01-009](#)).

Scanning the dependencies of the SDK uncovered that the SDK imports several modules/packages having publicly known vulnerabilities, as summarized in [NOD-01-015](#). It must be noted that even though those vulnerabilities seem to be not directly reachable, it remains important to keep all dependencies up-to-date and upgrade them to versions including patches for vulnerabilities as soon as possible.

In summary, the SDK is in a fragile state from a security perspective. Even though no issues having *High* or *Critical* severity were discovered, the number of issues identified during the engagement is concerning. The audit discovered several areas exposed to risk - such as conversion/rounding issues or attacks mountable by rogue upstream services (Denial-of-Service, request forgery).

These areas give rise to worry, as applications using the SDK rely on number conversions implemented in the SDK to obtain correct information about funds. Since the connection configuration options appear insufficient, they preclude a secure configuration from appearing in upstream services.

Moving forward, Cure53 strongly advises addressing all identified vulnerabilities to close the existing attack surface of the SDK. Tackling all the miscellaneous issues will further equip the SDK with defense-in-depth mechanisms and measures. In terms of a broad outlook, it is recommended to perform audits of the SDK on a regular basis, especially as the functionality of the codebase grows.

Cure53 would like to thank Ales Puchilo, Alex, Artem, Kiryl Laptanovich and Stan from the Noders LLC team for their excellent project coordination, support and assistance, both before and during this assignment.